

Git - Local

Working Locally

- working directory
- index (staging area)
- local repo
- remote reference
- Remote (GitHub)

Starting from scratch

- Create a new directory
- `git init`
- This creates a new git repo
 - all of the info on git is stored in `.git`

Tracking our changes

- we create our files
- stage the changes
 - `git add [filename]`

Are our changes saved?

- no, just staged
- `git commit -m 'Created [filename]'`

Why do we have a staging area?

- imagine making a modification to our file
 - We hate it
- We don't want to push all changes

Detecting changes

- your new friend
 - git status
- git status tells you which files:
 - have been added/changed since last commit
 - are not tracked
- We can see how a file has changed since last commit:
 - git diff
 - git diff [file]

Other operations

- `git mv` - rename or move
- `git rm` - remove
- `git log` - this will show you the history of commits in our repo
- SHA-1 hash used to identify commits

Oops

- How do we correct mistakes?
 - `git restore --staged file` - unstages file
 - `git reset file` - old way
- wait want to get rid of all my changes to a file!
 - `unstage`
 - `git restore file`
 - `git checkout file` - old way
- OH NO!
 - don't delete untracked files (from top level of repo)
 - `git restore --staged --worktree .`
 - or reset changes and possibly delete untracked files (CAREFUL)
 - `git reset --hard`

Cheatsheet

- Main file manipulation git commands:
 - git init - initialize git repo in folder
 - git add file - Add files/changes to staging area
 - git mv source dest - move/rename file and stage
 - git rm file - delete file and stage
 - git commit -m "message" - commit staged changes to local repo
- Status Commands
 - git status - view which files changed since last commit
 - git diff file - view how file changed since last commit
 - git diff - view how all files changed
 - git log - view list of commits